Innovators@DravidianCodeMix 2026: A Unified Transformer-Based Approach to Hate and Hope Speech Detection in Code-Mixed Dravidian Languages

Harish Sri Ragunath V, Jaiyanth T, Hari Prasath G

Innovators

FIRE 2026, DravidianCodeMix Shared Task

Abstract

This paper presents the Innovators team's participation in DravidianCodeMix 2026 across three task settings: Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. We develop multilingual Transformer-based classifiers using XLM-R encoders with task-specific prediction heads. The Tamil-English system uses a hierarchical classifier for five annotation levels, the Tulu system jointly predicts coarse- and fine-grained hope-speech labels, and the Kannada system performs binary hate-speech classification. The training pipeline includes supervised fine-tuning, label encoding, class-imbalance handling where applicable, AdamW optimization, learning-rate scheduling, gradient clipping, mixed-precision GPU training, early stopping, checkpointing, and a reproducible prediction pipeline. In the official DravidianCodeMix 2026 leaderboard, Innovators ranked second in Subtask 1 with an overall Macro-F1 of 0.4394, first in Subtask 2 with a Macro-F1 of 0.4348 and accuracy of 0.5121, and third in Subtask 3 with a Macro-F1 of 0.7652. These results demonstrate the effectiveness of adapting a common multilingual Transformer foundation to different annotation structures and linguistic settings.

Keywords: hate speech detection; hope speech detection; code-mixed text; Dravidian languages; multilingual Transformers; XLM-R; hierarchical classification; multi-task learning

1. Introduction

Online communication in Dravidian languages frequently combines native scripts, English, transliterated text, informal spelling, emojis, abbreviations, and context-dependent expressions. These characteristics make automatic detection of harmful and supportive speech substantially more difficult than conventional monolingual text classification. The same semantic expression can appear in multiple scripts or spelling variants, while short social-media comments may provide limited context.

DravidianCodeMix 2026 provides a shared evaluation setting for hate and hope speech detection across Dravidian languages. Our team participated in three task settings: Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. We use a common multilingual Transformer foundation while adapting the output architecture to each task: hierarchical prediction for Tamil-English, joint coarse/fine prediction for Tulu, and binary classification for Kannada.

The main contributions of this working note are: (1) a unified multilingual Transformer-oriented workflow covering all three subtasks; (2) task-specific classification heads adapted to different label structures; (3) class-imbalance-aware training for the Kannada task; (4) validation using accuracy, macro-F1, weighted-F1, and the task-specific joint score; (5) systematic debugging of training and prediction behavior; and (6) a reproducible model-checkpoint and submission-generation pipeline.

2. Task Description

2.1 Tamil-English Hate Speech

The Tamil-English setting addresses hate speech in code-mixed user-generated text. The data may contain Tamil in native script, English, Romanized Tamil, or mixtures of these forms. The model predicts the categories specified by the shared-task hierarchical annotation scheme. The five-level classifier uses a shared XLM-R representation and separate classification layers for the five label levels.

2.2 Tulu Hope Speech

The Tulu setting focuses on identifying hope or supportive speech. The system predicts both coarse-grained and fine-grained hope-speech labels using a shared multilingual representation and two task-specific classification heads. The development data used in the reported run contains 3,513 training samples and 753 validation samples, with three coarse classes and five fine classes.

2.3 Kannada Hate Speech

The Kannada setting focuses on binary hate speech classification. The reported experiment contains 3,040 training samples and 500 validation samples. The training labels contain 1,687 instances of label 0 and 1,353 instances of label 1, while the validation labels contain 288 instances of label 0 and 212 instances of label 1. The distribution motivates attention to balanced training and macro-F1.

3. Dataset and Preprocessing

The task datasets are loaded from CSV files and unnecessary index columns are removed when present. Labels are converted to integer identifiers using LabelEncoder objects fitted on the training labels. The fitted encoders are serialized so that integer predictions can later be converted back to the original task labels during submission generation.

Comments are converted to strings and tokenized with the tokenizer associated with the selected XLM-R model. Input sequences are truncated to the configured maximum length and padded as required by each training pipeline. Attention masks are supplied to the encoder to distinguish actual tokens from padding. The same tokenizer/model family and compatible preprocessing are used during prediction.

The preprocessing intentionally avoids aggressive normalization. This preserves script identity, transliteration patterns, punctuation, emojis, and informal spelling that may contain useful classification information. Potential improvements include transliteration-aware normalization, data augmentation, spelling normalization, and multilingual error analysis.

4. Methodology

4.1 Multilingual Transformer Encoder

The main modelling strategy uses FacebookAI/xlm-roberta-large for the Tamil-English and Tulu experiments and FacebookAI/xlm-roberta-base for the reported Kannada experiment. XLM-R produces contextual token representations that are then passed to task-specific classification components. The pretrained encoders are fine-tuned rather than used only for zero-shot prediction.

4.2 Representation and Classification Head

For Tamil-English, a hierarchical classifier uses a shared encoder representation and five classification layers with output sizes of 3, 6, 3, 32, and 7. For Tulu, a shared feed-forward representation is connected to a three-class coarse head and a five-class fine head. For Kannada, the classifier uses dropout, a linear projection, ReLU, a second dropout layer, and a final two-class linear layer. Predictions are obtained from the highest output logit.

4.3 Class-Imbalance Handling

Balanced class weights are explicitly used in the Kannada training pipeline with weighted cross-entropy loss. This reduces the tendency to favor the more frequent class and makes macro-F1 a useful diagnostic for minority-class behavior. The Tulu multi-task setting optimizes the coarse and fine objectives jointly, while the Tamil-English model optimizes its hierarchical outputs.

4.4 Optimization

The models are fine-tuned using AdamW. The training pipelines use learning-rate scheduling with warm-up/decay, gradient clipping, checkpointing, and early stopping based on validation performance. The reported runs use GPU acceleration and automatic mixed precision where configured.

4.5 Mixed Precision

Automatic mixed precision is used where configured to reduce GPU memory consumption and improve training efficiency. The reported Kannada run used the PyTorch AMP interface available in the experimental environment.

5. Experimental Setup

Table 1: Experimental configuration and dataset sizes for the three reported subtasks.

Subtask	Model / Configuration
Subtask 1	XLM-R large; hierarchical heads (3/6/3/32/7); 8 epochs
Subtask 2	XLM-R large; shared representation + coarse/fine heads; 10 epochs
Subtask 3	XLM-R base; max length 128; batch 8; LR 5×10^{-5}; weighted CE; AdamW; 10 epochs

The configurations in Table 1 summarize the reported systems used for the three subtasks. Dataset-specific training details are provided in the corresponding task descriptions and the public repository.

Evaluation was performed using metrics appropriate to the task structure. Macro-F1 was emphasized because it gives equal importance to classes and is therefore informative under class imbalance. Weighted-F1 was also monitored during development to reflect performance under the observed class distribution. For Subtask 1, the official score is the mean Macro-F1 across the five hierarchical levels. For Subtask 2, the official leaderboard reports the submitted run using the shared-task evaluation, while the local experiments additionally tracked coarse accuracy, fine accuracy, Macro-F1, Weighted-F1, and the joint score. For Subtask 3, the primary evaluation measure is Macro-F1, with precision and recall used to inspect the balance between the two hate-speech classes. The distinction between validation metrics and official leaderboard metrics is maintained throughout the paper.

5.2 Evaluation Protocol

These configurations were selected to match the different annotation structures rather than forcing one classifier onto all tasks. The Tamil-English system explicitly preserves the hierarchy by conditioning each subsequent prediction head on the shared representation and the previous level output. The Tulu system shares the representation between coarse and fine objectives, allowing the two label granularities to be learned jointly. The Kannada system instead uses a single binary head, with class weighting introduced because the training distribution is not perfectly balanced. Across the experiments, validation metrics were monitored and the best checkpoint according to the relevant model-selection criterion was retained for prediction.

The three subtasks differ in both dataset size and label structure. The Tamil-English hate speech setting used 4,255 training instances and 532 validation instances. Its hierarchical classifier predicted five label levels with output dimensions of 3, 6, 3, 32, and 7, and training was conducted for eight epochs. The Tulu hope speech setting used 3,513 training instances and 753 validation instances. It employed a shared XLM-R large representation with separate coarse and fine prediction heads containing three and five classes, respectively, and was trained for ten epochs. The Kannada hate speech setting used 3,040 training instances and 500 validation instances, with 1,687 training examples belonging to class 0 and 1,353 to class 1. The Kannada system used XLM-R base and a two-class feed-forward classifier, trained for ten epochs with AdamW and weighted cross-entropy; the reported configuration used a maximum sequence length of 128, batch size 8, and learning rate 5×10^{-5}.

5.1 Dataset Sizes and Training Configuration

6. Results

The final results reported in this section are the team's official DravidianCodeMix 2026 leaderboard results. Development/validation results were used during model selection and debugging, whereas the values below correspond to the submitted systems evaluated by the shared-task organizers. Subtask 1 is ranked by the mean Macro-F1 across Levels 1–5; Subtask 2 reports the official Macro-F1 for the submitted run together with accuracy; and Subtask 3 is ranked by Macro-F1.

Table 2: Official leaderboard performance across the three DravidianCodeMix 2026 subtasks.

Subtask	Language / Setting	Accuracy	Macro-F1	Official / Final
Subtask 1	Tamil-English Hate Speech	—	0.4394	Official / Rank 2
Subtask 2	Tulu Hope Speech	0.5121	0.4348	Official / Rank 1
Subtask 3	Kannada Hate Speech	—	0.7652	Official / Rank 3

Table 3: Official level-wise Macro-F1 for Subtask 1 (Tamil-English Hate Speech).

Subtask 1	L1 Macro-F1	L2 Macro-F1	L3 Macro-F1	L4 Macro-F1	L5 Macro-F1	Overall Macro-F1
Tamil-English Hate Speech	0.5403	0.5231	0.6729	0.1048	0.3560	0.4394

The official results differ from the intermediate validation results recorded during development. In particular, the early Kannada run that predicted class 1 for most or all validation examples produced an artificially poor validation Macro-F1 and was subsequently debugged; it is not used as the competition result. The corrected training pipeline produced a substantially stronger development result, and the final submitted Kannada system achieved 0.7652 Macro-F1 on the official leaderboard. The final competition values in Table 2 therefore take precedence over intermediate validation values when describing team performance.

Leaderboard summary: Innovators achieved Rank 2 in Subtask 1, Rank 1 in Subtask 2, and Rank 3 in Subtask 3. Across the three official task settings, the systems therefore remained within the top three submissions for every subtask.

For Subtask 1, Innovators obtained level-wise official Macro-F1 values of 0.5403, 0.5231, 0.6729, 0.1048, and 0.3560 for Levels 1–5, respectively, giving an overall Macro-F1 of 0.4394 and second place on the leaderboard. For Subtask 2, the submitted system achieved 0.5121 accuracy and 0.4348 Macro-F1, ranking first. For Subtask 3, the submitted Kannada system achieved 0.7652 Macro-F1, with 0.7653 precision and 0.7679 recall, ranking third.

Taken together, the official results support the central design choice of sharing a multilingual Transformer foundation while tailoring the prediction architecture to each annotation scheme. However, the ranking differences also show that a single architecture and training recipe cannot guarantee uniform performance across languages and task structures. Future experiments should therefore focus on task-specific learning-rate and sequence-length tuning, label-aware sampling, transliteration-aware preprocessing, and targeted error analysis for low-performing hierarchical levels.

The Kannada system achieved 0.7652 Macro-F1 and ranked third. The submitted architecture uses XLM-R base followed by a feed-forward classification head and weighted cross-entropy to address class imbalance. The development experiments were particularly useful for identifying an early prediction-collapse problem in which the model favored one class. After correcting the training and validation pipeline, the model reached substantially stronger development performance, and the final submitted system performed competitively on the official test set. This emphasizes the importance of checking prediction distributions and validating the complete evaluation loop when Macro-F1 is the principal metric.

The Tulu system achieved 0.5121 accuracy and 0.4348 Macro-F1 in the official evaluation and ranked first. The model jointly predicts three coarse classes and five fine-grained classes from a shared XLM-R representation. Joint prediction allows the two label spaces to share contextual information while retaining separate task-specific output heads. The difference between the development joint score and the final leaderboard score also illustrates why final competition results should be reported separately from validation experiments.

The Tamil-English result is measured as the mean Macro-F1 over five hierarchical levels. The level-wise scores show that the system performs relatively well at Levels 1–3, with Macro-F1 values of 0.5403, 0.5231, and 0.6729. In contrast, Level 4 reaches only 0.1048 Macro-F1, while Level 5 reaches 0.3560. The overall Macro-F1 of 0.4394 therefore reflects substantial variation among the annotation levels. The low Level-4 score is an important limitation of the submitted system and indicates that deeper hierarchical distinctions remain difficult, possibly because the relevant categories are more fine-grained or less evenly represented.

The official leaderboard results provide a clearer picture of the submitted systems than the intermediate validation checkpoints. Innovators obtained a second-place finish in the Tamil-English hate-speech setting, a first-place finish in the Tulu hope-speech setting, and a third-place finish in the Kannada hate-speech setting. This consistent top-three performance across heterogeneous task definitions suggests that the use of a multilingual pretrained encoder together with task-specific output structures was effective for adapting to the three subtasks.

6.1 Official Leaderboard Analysis

7. Error Analysis and Observations

The Kannada experiments exposed an important failure mode in which the classifier initially favored one class. Logit inspection showed a consistent preference for class 1, and an early validation report produced zero precision, recall, and F1 for class 0. This behavior motivated checks of the label encoder, class distributions, optimizer updates, training-loop control flow, and prediction accumulation.

A particularly important implementation issue was an unintended break inside the training loop. Such a break terminates an epoch after a single batch and prevents the model from receiving the intended number of optimization updates. Another issue occurred when the loop was written as 'for step, batch in train_loader'; a DataLoader returns each batch directly, so enumerate(train_loader) is required when an explicit step index is needed. Validation predictions and targets must also be accumulated over the complete validation set before metrics are computed.

The three task settings exhibit different sources of difficulty. Tamil-English contains code switching, transliteration, spelling variation, sarcasm, and implicit abuse. Tulu presents a low-resource multilingual setting with both coarse and fine-grained labels. Kannada includes short comments, lexical overlap, class imbalance, and ambiguous or context-dependent expressions. These factors can lead to confusion between harmful, neutral, and supportive language.

The official results show that performance varies substantially with the structure and linguistic demands of each task. Subtask 1 achieved strong performance at Levels 1–3 but a much lower Level-4 Macro-F1 of 0.1048, indicating difficulty with one of the deeper hierarchical categories. Subtask 2 achieved the highest team rank despite the challenges of joint coarse- and fine-grained prediction. Subtask 3 produced the strongest Macro-F1 among the three Innovators systems, although it still trails the leading submissions. These results reinforce the importance of task-specific heads, balanced evaluation, and careful validation when working with code-mixed and low-resource Dravidian-language data.

8. Prediction and Submission Pipeline

After training, the best model checkpoint and label encoder(s) are retained. The prediction script loads the same tokenizer, encoder architecture, classifier structure, and saved parameters used during training. Test comments are tokenized with the same preprocessing configuration, and predictions are generated by selecting the maximum output logit for each task head.

Integer predictions are converted back to the original labels using the saved encoders. The resulting task-specific prediction files must follow the current DravidianCodeMix 2026 submission requirements for filenames, columns, delimiter, and packaging. The supplied Kannada workflow produces a task-specific prediction TSV/CSV artifact and the team's submission ZIP contains the individual language outputs required by the platform.

Before submission, the prediction pipeline should verify that the number of predictions equals the number of test instances, every required ID occurs exactly once, labels belong to the permitted label set, no training or validation rows are included, and the ZIP archive contains only the required submission files.

9. Reproducibility

The implementation uses Python, PyTorch, Hugging Face Transformers, pandas, NumPy, and scikit-learn. The reported Kannada workflow fixes the random seed at 42. Label encoders are serialized for reproducible label decoding, and the best model parameters are saved as task-specific checkpoint files. The Tamil-English and Tulu runs likewise retain their trained model/checkpoint and label-encoding artifacts.

The implementation, training scripts, prediction scripts, and experimental results for all three subtasks are publicly available in the team repository.

GitHub repository: https://github.com/Harish-sri-ragunath-V/Innovators-DravidianCodeMix-2026

Several limitations remain. First, the systems rely on a pretrained multilingual encoder and do not explicitly model transliteration equivalence, spelling normalization, or language-specific phonological variation. This is relevant for code-mixed social-media text, where the same expression may occur in multiple scripts or highly variable Romanized forms. Second, the hierarchical Tamil-English system shows a marked reduction in Macro-F1 at Level 4, indicating that deeper and more fine-grained distinctions remain difficult. Third, the Tulu task is constrained by its relatively low-resource setting and the need to distinguish both coarse and fine hope-speech categories. Finally, the Kannada experiments demonstrated that apparently strong accuracy can coexist with weak minority-class behavior when prediction distributions are not inspected carefully. These observations motivate more systematic error analysis and calibration in future experiments.

9.1 Limitations

10. Conclusion

This working note presents a unified Transformer-based participation in DravidianCodeMix 2026 across Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. XLM-R provides a common multilingual representation while task-specific heads accommodate hierarchical, multi-task, and binary classification settings. The official leaderboard results show that Innovators ranked second in Subtask 1, first in Subtask 2, and third in Subtask 3, demonstrating competitive performance across all three settings. The results also highlight the difficulty of deeper hierarchical labels and the benefit of task-specific modelling for heterogeneous annotation schemes.

Future work includes transliteration-aware preprocessing, data augmentation, task-specific hyperparameter tuning, parameter-efficient fine-tuning, ensemble methods, threshold calibration, and systematic multilingual error analysis. These improvements may be particularly useful for low-resource languages, highly code-mixed comments, and examples where harmful or supportive intent is expressed indirectly.

11. AI Writing Assistance Disclosure

AI-based writing and coding assistance was used during preparation of this working note and implementation workflow. The authors reviewed and edited the generated material and remain responsible for verifying all technical statements, experimental results, references, code, and the final submission. Any generated material retained in the final paper should be checked against the team's actual experiments and the official DravidianCodeMix 2026 instructions.

12. Copyright Clause

© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

The authors should complete and submit any separate author agreement required by the DravidianCodeMix 2026 and CEUR-WS organizers.

Acknowledgments

We thank the DravidianCodeMix 2026 organizers and the FIRE community for providing the shared-task resources and evaluation framework. No institutional information is included here in accordance with the double-blind submission guidance.

References

[1] DravidianCodeMix 2026 Organizers. DravidianCodeMix 2026: Shared Task on Hate and Hope Speech Detection in Dravidian Languages. FIRE 2026.

[2] Sripriya N., Bharathi Raja Chakravarthi, Thenmozhi Durairaj, Bharathi B., Prasanna Kumar Kumaresan, Subalalitha C. N., Anusha M. D., Parameshwar R. Hegde, and Deepthi Vikram. Overview of the shared task on Offensive Language Identification in Dravidian code-Mixed Languages. Working Notes of FIRE 2025, 2025, pp. 225–233.

[3] Chakravarthi, B. R., Priyadharshini, R., Thavareesan, S., Chinnappa, D., Thenmozhi, D., Ponnusamy, R., et al. DravidianCodeMix: Sentiment Analysis and Offensive Language Identification Dataset for Dravidian Languages in Code-Mixed Text. Language Resources and Evaluation, 56, 765–806, 2022.

[4] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., and Stoyanov, V. Unsupervised Cross-lingual Representation Learning at Scale. Proceedings of ACL, 2020.

[5] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692, 2019.

[6] Loshchilov, I. and Hutter, F. Decoupled Weight Decay Regularization. International Conference on Learning Representations (ICLR), 2019.